Interrogation Avancée en SQL : Jointures Relationnelles et Regroupements (GROUP BY & HAVING)

Thème principal : Bases de données relationnelles et requêtes SQL complexes

Synthèse exécutive : Ce cours de la ressource R1.05 aborde les mécanismes fondamentaux du langage SQL pour croiser des données issues de multiples tables relationnelles via les jointures (INNER JOIN, LEFT JOIN) et structurer les résultats par groupes au moyen des clauses GROUP BY et HAVING. L'objectif est d'assimiler la logique d'exécution des requêtes complexes, de maîtriser le calcul d'agrégats statistiques (AVG, COUNT, SUM) et d'éviter les pièges classiques tels que la génération involontaire de produits cartésiens ou l'utilisation erronée de fonctions d'agrégation dans la clause WHERE.

Les Jointures Relationnelles : Interrogation Multi-Tables (INNER JOIN vs LEFT JOIN)

Dans un Système de Gestion de Base de Données Relationnel (SGBDR), les données ne sont jamais stockées dans un tableau unique afin d'éviter les anomalies de mise à jour et la redondance d'information. La normalisation répartit les concepts métiers dans des tables distinctes. La jointure est l'opération algébrique indispensable pour recomposer les informations au moment de la requête. Lors de l'évaluation du bloc FROM, le moteur de base de données associe les tables selon l'attribut pivot (généralement la clé étrangère d'une table pointant vers la clé primaire d'une autre). L'utilisation explicite de la clause INNER JOIN garantit que seules les paires valides apparaîtront dans le résultat. En revanche, si l'on souhaite procéder à un inventaire exhaustif sans perdre les éléments sans correspondance (par exemple, tous les étudiants, y compris ceux sans notes), le LEFT JOIN est indispensable : les attributs manquants de la table de droite sont alors comblés par des valeurs 'NULL'. Omis ou mal configuré, le prédicat de liaison génère un produit cartésien très lourd qui surcharge la mémoire et le processeur du SGBD.

- Normalisation & Équijointure : Séparation des entités pour éviter la redondance et création de liaisons explicites via Clé Primaire (PK) et Clé Étrangère (FK).
- INNER JOIN (Jointure Interne) : Restitue exclusivement les combinaisons de n-uplets satisfaisant le prédicat d'association défini dans la clause ON.
- LEFT JOIN (Jointure Externe Gauche) : Conserve la totalité des enregistrements de la table de gauche et complète par NULL si aucun enregistrement correspondant n'existe dans la table de droite.
- Produit Cartésien (CROSS JOIN) : Résultat d'une jointure sans critère de liaison (clause ON omise), générant une combinaison de toutes les lignes de la table A avec toutes celles de la table B (N x M lignes).

Cas concret enseignant : *Le professeur illustre la différence à travers deux cas réels de l'IUT de Bayonne : 1. L'INNER JOIN fonctionne comme le portique de sécurité du campus : seuls les étudiants possédant un badge actif ET une inscription valide peuvent franchir la porte. Si l'un des deux éléments manque, l'accès est refusé (la ligne est exclue). 2. Le LEFT JOIN équivaut à la liste d'appel officielle du secrétariat : tous les étudiants inscrits sur le registre de l'IUT y figurent obligatoirement. Si un étudiant n'a pas encore de note attribuée, son nom reste affiché et la case note contient la mention 'NULL'.*

Regroupement de Données et Agrégation : Clauses GROUP BY et HAVING

L'agrégation permet de répondre à des problématiques décisionnelles (calcul de moyennes, sommes de ventes, décomptes d'effectifs). Dès qu'un regroupement est initié via la clause GROUP BY, la table d'origine se retrouve partitionnée en sous-ensembles disjoints. Une erreur courante en évaluation consiste à tenter de filtrer un calcul statistique dans la clause WHERE (par exemple 'WHERE AVG(note) > 10'). Cette écriture est physiquement rejetée par le processeur de requêtes SQL. En effet, la clause WHERE est évaluée ligne par ligne au niveau de la table initiale, bien avant que les groupes ne soient formés et que les fonctions d'agrégation ne soient calculées. Pour appliquer un critère de restriction sur le résultat d'un calcul agrégé, SQL met à disposition la clause HAVING, qui

intervient exclusivement après la phase de regroupement des tuples.

• Fonctions d'Agrégation (COUNT, SUM, AVG, MIN, MAX) : Calculs statistiques scalaires réduisant un ensemble de n-uplets à une valeur synthétique unique.

• Clause GROUP BY : Subdivise les n-uplets filtrés par la clause WHERE en sous-groupes homogènes partageant les mêmes valeurs pour les colonnes spécifiées.

• Règle d'or du SELECT avec GROUP BY : Toute colonne présente dans la clause SELECT doit obligatoirement figurer dans la clause GROUP BY ou être intégrée dans une fonction d'agrégation.

• Clause HAVING : Prédicat de filtrage s'appliquant spécifiquement sur les sous-groupes formés et sur le résultat des fonctions agrégées.

• Ordre logique d'exécution SQL : FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY.

Cas concret enseignant : *L'enseignant prend l'exemple du dépouillement physique des copies d'examen dans l'amphithéâtre de l'IUT : 1. GROUP BY : Le professeur répartit toutes les copies physiques dans trois bannettes distinctes selon le département (Informatique, GEA, TC). 2. Agrégation (AVG) : Pour chaque bannette, il calcule la note moyenne du groupe. 3. HAVING : Si le directeur impose d'afficher uniquement les paquets dont la moyenne globale dépasse 12/20, c'est la clause HAVING qui élimine les bannettes non conformes. La clause WHERE, elle, aurait servi avant le tri à jeter les copies d'étudiants fraudeurs.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il strictement impossible d'écrire 'WHERE AVG(note) > 10' dans une requête SQL ?
Réponse du professeur : Parce que la clause WHERE s'exécute chronologiquement au niveau des lignes individuelles avant même que le regroupement (GROUP BY) et les fonctions d'agrégation (AVG) ne soient calculés. Le moteur SQL ne connaît pas encore la moyenne à l'étape du WHERE.

Question étudiant : Que se passe-t-il si j'oublie d'inclure dans la clause GROUP BY une colonne présente dans le SELECT ?
Réponse du professeur : Le SGBD lève une erreur de syntaxe ou d'agrégation (ex: 'colonne non valide dans la liste SELECT') car il ne saurait pas quelle valeur afficher parmi toutes celles du groupe pour cette colonne non regroupée.

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données relationnelle de l'IUT composée des tables Etudiant(id_etud, nom, id_promo) et Note(id_note, id_etud, matiere, note). Écrire la requête SQL qui affiche le nom de chaque étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants ayant au moins 2 notes enregistrées dans cette matière et une moyenne stricte supérieure à 12/20. Le résultat doit être trié par moyenne décroissante.

Démarche de résolution :
1. Identifier les tables sources et effectuer l'équijointure dans le FROM : FROM Etudiant E INNER JOIN Note N ON E.id_etud = N.id_etud. 2. Filtrer les enregistrements individuels pour ne conserver que la matière 'R1.05' via le WHERE : WHERE N.matiere = 'R1.05'. 3. Regrouper les données par identifiant et par nom d'étudiant avec GROUP BY : GROUP BY E.id_etud, E.nom. 4. Filtrer les groupes d'étudiants respectant les deux conditions d'agrégation dans le HAVING : HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12. 5. Définir la projection de sortie et nommer l'agrégat dans le SELECT : SELECT E.nom, AVG(N.note) AS moyenne. 6. Trier le résultat final par ordre décroissant de la moyenne via le ORDER BY : ORDER BY moyenne DESC.

```sql
/* ========================================================== */
/* TITRE : Requête SQL avec jointure, agrégation et filtres   */
/* AUTEUR : Étudiant BUT 1 Info - IUT Anglet                  */
/* ========================================================== */

// CONSTANTES
// Aucune constante numérique brute dans le code

// VARIABLES
// (Base de données) >> tables Etudiant et Note

// TRAITEMENTS
SELECT E.nom, AVG(N.note) AS moyenne
FROM Etudiant E
INNER JOIN Note N ON E.id_etud = N.id_etud
WHERE N.matiere = 'R1.05'
GROUP BY E.id_etud, E.nom
HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12
ORDER BY moyenne DESC;
```

Résultat : La requête retourne un jeu de résultats tabulaire affichant le nom des étudiants qualifiés ainsi que leur moyenne calculée en R1.05, triés du meilleur au moins bon.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'application entre la clause WHERE et la clause HAVING en SQL ?

R : La clause WHERE filtre les lignes individuelles avant le regroupement, tandis que la clause HAVING filtre les groupes formés après l'agrégation.

Q : Quel est le résultat produit par une requête avec LEFT JOIN si un enregistrement de la table de gauche ne possède aucune correspondance dans la table de droite ?

R : L'enregistrement de gauche est conservé dans le résultat et les colonnes issues de la table de droite prennent la valeur NULL.

Q : Si la table 'Etudiant' comporte 10 lignes et la table 'Note' comporte 5 lignes, combien de lignes retourne un produit cartésien (CROSS JOIN) entre ces deux tables ?

R : Il retourne 50 lignes (10 x 5), car chaque ligne de la première table est combinée avec toutes les lignes de la seconde.